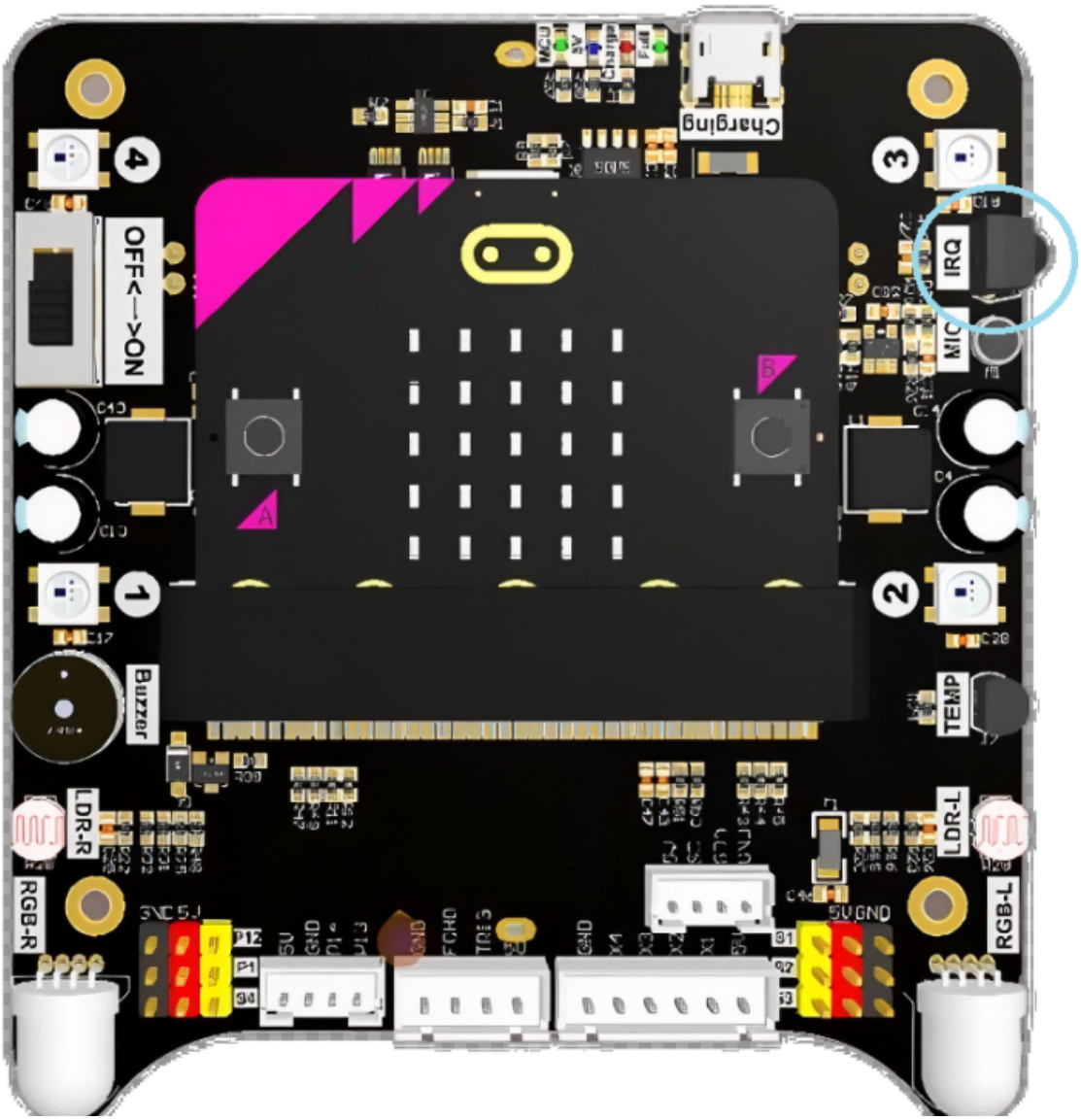
Infrared Remote

Feature Description

The Lastbit board is equipped with an infrared receiver module that can receive signals from the infrared remote control to control the car. In this section, we will learn to use MicroPython programs to implement infrared remote control.



Precautions

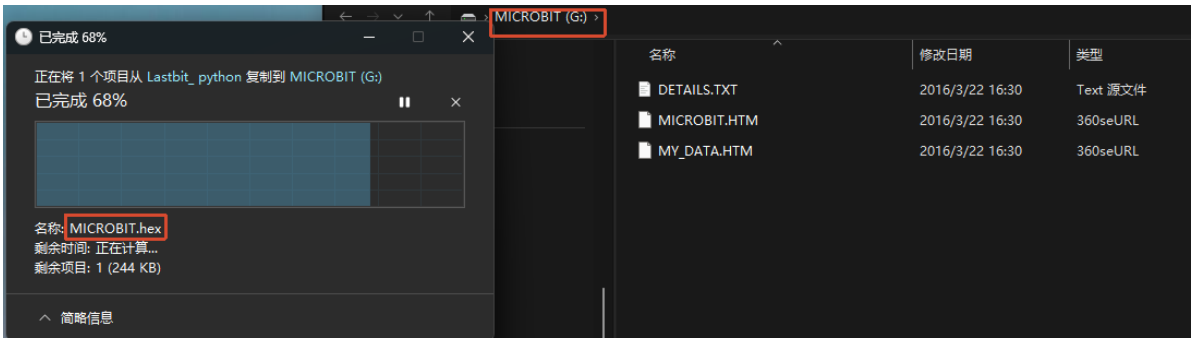
- Make sure the battery is fully charged. When using the infrared remote control, point the remote control at the infrared receiver head on the expansion board.
- There is a plastic tab at the bottom of the infrared remote control that needs to be removed before it can be used properly.
- The infrared light emitted by the infrared remote control is invisible to the human eye, but it can be seen under a camera without an infrared filter.

Infrared Remote Control Principle

Infrared remote control consists of two components: the transmitter and the receiver. At the transmitting end, a microcontroller encodes and modulates the binary signal to be sent into a series of pulse train signals, which are emitted as infrared signals through an infrared transmitting tube. The infrared receiver is responsible for receiving, amplifying, detecting, and shaping the infrared signal, and demodulating the remote control encoding pulses. To reduce interference, an integrated infrared receiver head HS0038B (receiving infrared signal frequency: 38 kHz) is used to receive the infrared signal. It simultaneously amplifies, detects, and shapes the signal to produce a TTL-level encoded signal, which is then sent to the microcontroller. The microcontroller decodes it to obtain the code value corresponding to each button and executes the relevant program to control the car.

Experimental Steps

Before flashing, make sure `LASTBIT.hex` has been flashed; otherwise, an error will occur and the lastbit control module cannot be imported.

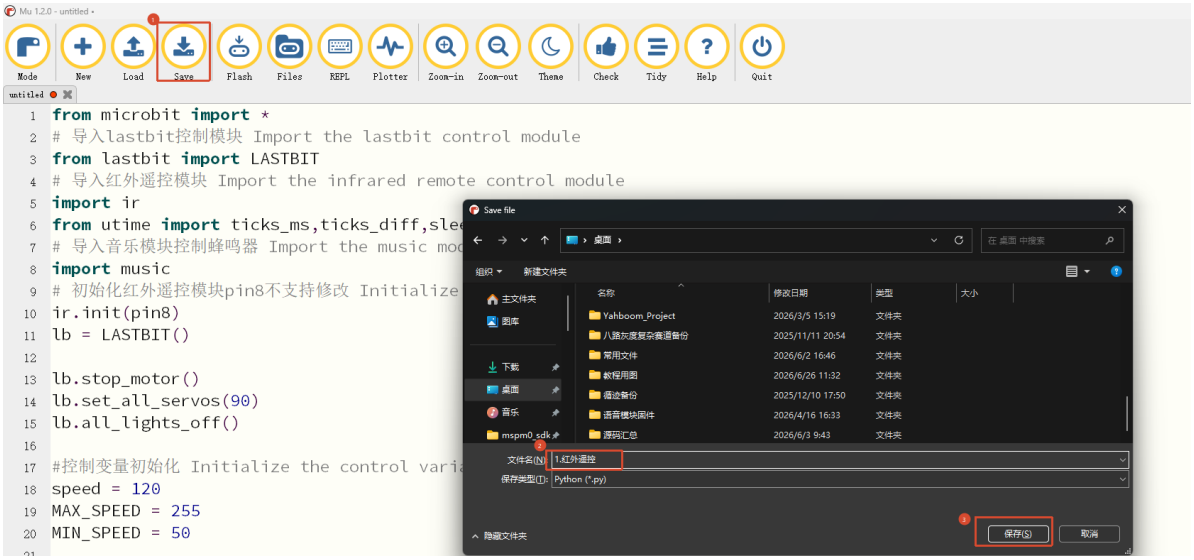


1. Before flashing, switch the switch to the OFF position and connect the computer to the microbit main board using a microUSB cable. **Note: it should be the Microbit interface, not the expansion board's Charging port.**
2. Open the Mu software. Here you can directly copy the program source code we provide; the operation steps are as follows. You can also enter the code yourself in the editor window. Note! All English characters and symbols should be entered in English input mode. Use the Tab key for indentation, and end the last line with a blank line.

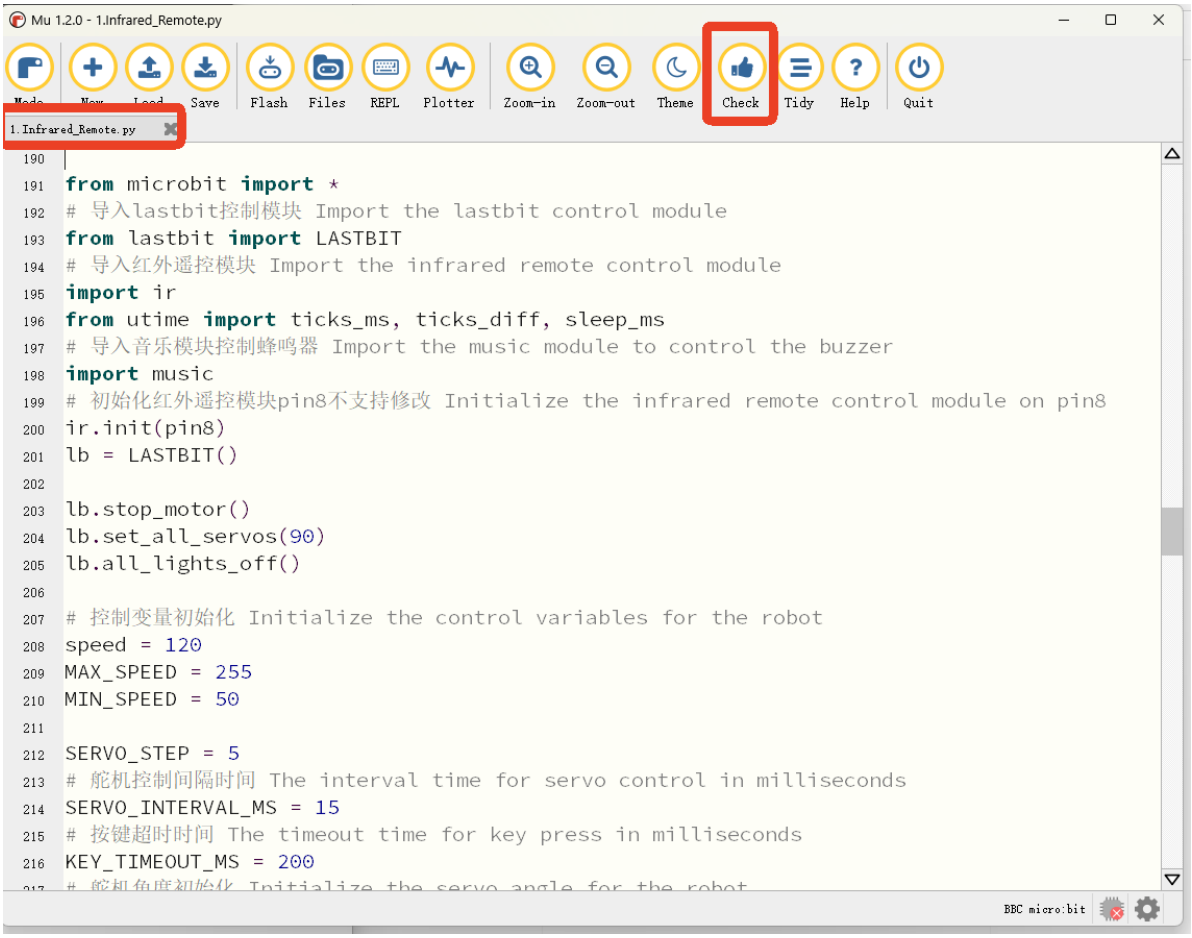
   Click the `New` button in the toolbar at the top left, and a file titled untitled will appear.

3. Copy the content of the `Code Implementation` section provided below into the current file. You will notice a small red dot after the title bar, indicating that the code content has been modified but is not yet saved.

Click the `Save` button in the toolbar above to save the current program to a local file. You can choose the save path yourself; here we save it directly to the desktop. The file is named after `Infrared_Remote` as an example.

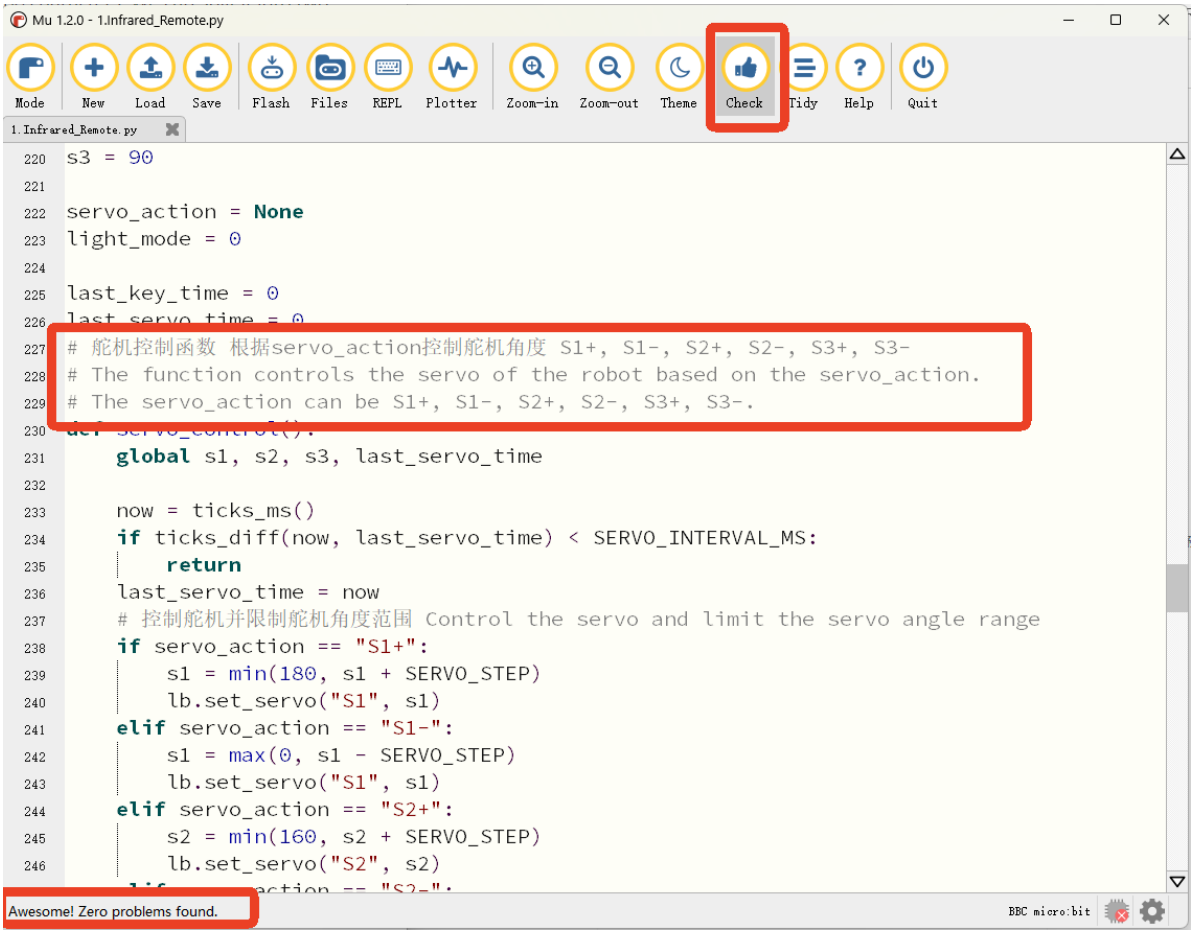


After saving, the red dot in the title bar will disappear. At this point, we can check whether the code has any syntax errors.



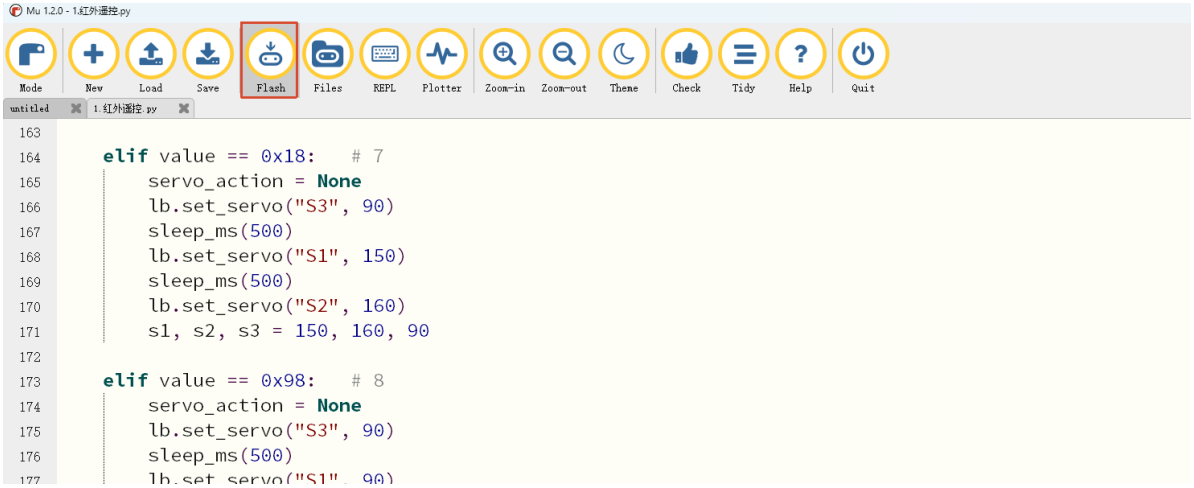
The blue box here is the compiler's warning, telling us that the current line is too long. This is because we have written quite a lot of Chinese and English comments, exceeding 88 characters. We can split it into two or more lines.

After modification, click `Check` again to check our program. The editor will display **Awesome!Zero problems found.** at the bottom, meaning no syntax errors were detected.



4. At this point, if you have connected the MICROBIT to the PC in advance following the steps above and have flashed `LASTBIT.hex`, the next step is to download the current program to the Microbit.

Click the `Flash` tool in the toolbar to start flashing.



During flashing, the orange-yellow indicator light near the Microbit main board will blink frequently, and the editor will show `Copied code onto micro:bit.` at the bottom, indicating that the flashing is complete. After flashing, the indicator light will no longer blink.

5. After flashing is complete, unplug the Microusb cable. At this time, the car is in a power-off state. Switch the switch to the (ON) position.

Code Implementation

```
from microbit import *
#  Import the lastbit control module
from lastbit import LASTBIT
#  Import the infrared remote control module
import ir
from utime import ticks_ms,ticks_diff,sleep_ms
#  Import the music module to control the buzzer
import music
#  Initialize the infrared remote control module on pin8
ir.init(pin8)
lb = LASTBIT()

lb.stop_motor()
lb.set_all_servos(90)
lb.all_lights_off()

# Initialize the control variables for the robot
speed = 120
MAX_SPEED = 255
MIN_SPEED = 50

SERVO_STEP = 5
#  The interval time for servo control in milliseconds
SERVO_INTERVAL_MS = 15
```

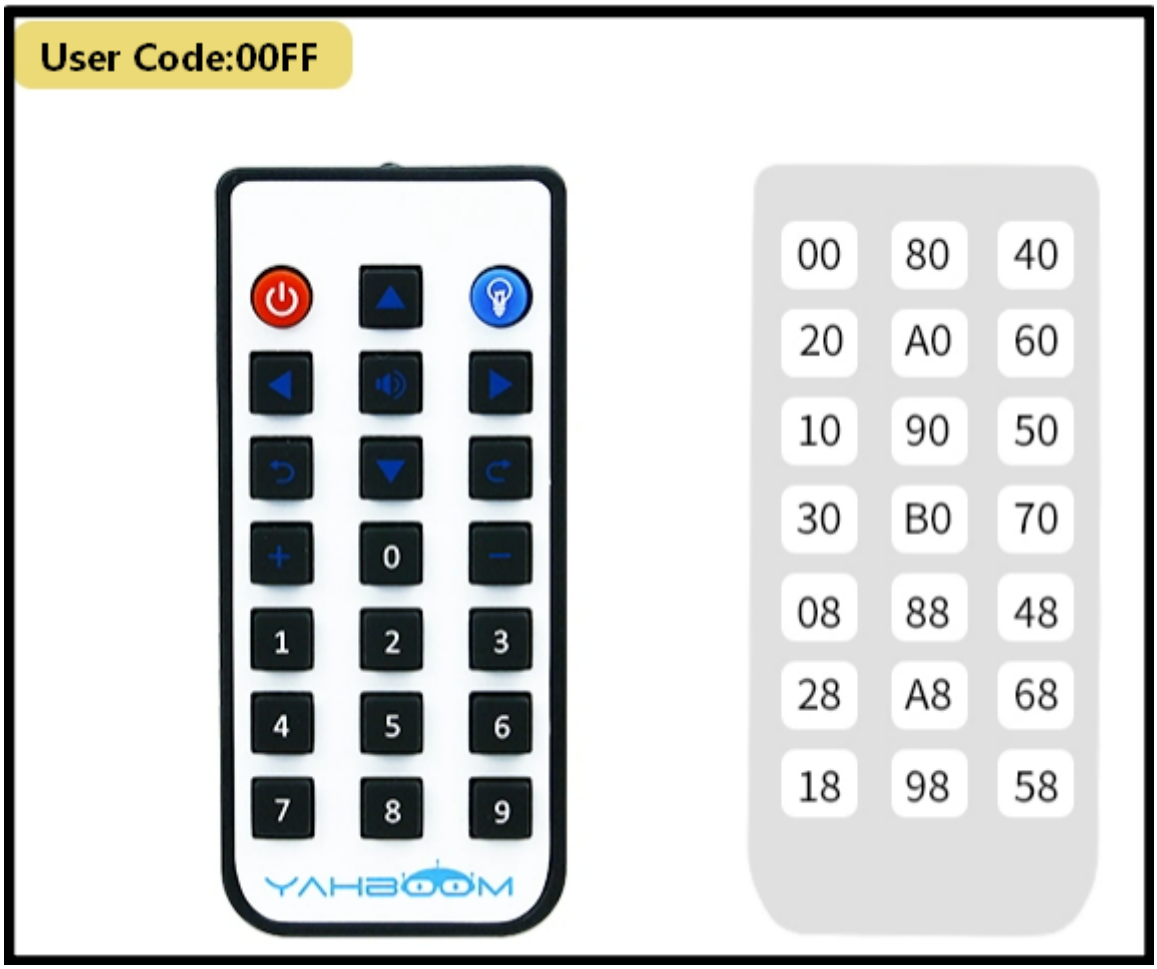
```
#  The timeout time for key press in milliseconds
KEY_TIMEOUT_MS = 200
# Initialize the servo angle for the robot
s1 = 90
s2 = 90
s3 = 90

servo_action = None
light_mode = 0

last_key_time = 0
last_servo_time = 0
#  The function controls the servo of the robot based on the servo_action.
# The servo_action can be S1+, S1-, S2+, S2-, S3+, S3-.
def servo_control():
    global s1, s2, s3, last_servo_time

    now = ticks_ms()
    if ticks_diff(now, last_servo_time) < SERVO_INTERVAL_MS:
        return
    last_servo_time = now
    #Control the servo and limit the servo angle range
    if servo_action == "S1+":
        s1 = min(180, s1 + SERVO_STEP)
        lb.set_servo("S1", s1)
    elif servo_action == "S1-":
        s1 = max(0, s1 - SERVO_STEP)
        lb.set_servo("S1", s1)
    elif servo_action == "S2+":
        s2 = min(160, s2 + SERVO_STEP)
        lb.set_servo("S2", s2)
    elif servo_action == "S2-":
        s2 = max(60, s2 - SERVO_STEP)
        lb.set_servo("S2", s2)
    elif servo_action == "S3+":
        s3 = min(180, s3 + SERVO_STEP)
        lb.set_servo("S3", s3)
    elif servo_action == "S3-":
        s3 = max(0, s3 - SERVO_STEP)
        lb.set_servo("S3", s3)


while True:
    now = ticks_ms()
    # Initialize the infrared remote control module and read the key value
    value = ir.read(pin8)
    value = value >> 8

    #  -1 means key release
    if value == -0x01:
        if ticks_diff(now, last_key_time) > KEY_TIMEOUT_MS:
            lb.stop_motor()
            servo_action = None
            display.clear()
        servo_control()
        continue
```

```
    last_key_time = now

    # Key value judgment
    if value == 0x80:      # ↑
        lb.set_motor(speed, speed)
        display.show(Image.ARROW_S)
    elif value == 0x90:    # ↓
        lb.set_motor(-speed, -speed)
        display.show(Image.ARROW_N)
    elif value == 0x20:    # ←
        lb.set_motor(0, speed)
        display.show(Image.ARROW_E)
    elif value == 0x60:    # →
        lb.set_motor(speed, 0)
        display.show(Image.ARROW_W)
    elif value == 0x10:    # Left rotation
        lb.set_motor(-speed, speed)
        display.show(Image.SQUARE_SMALL)
    elif value == 0x50:    # Right rotation
        lb.set_motor(speed, -speed)
        display.show(Image.DIAMOND_SMALL)

    elif value == 0x30:    # +
        speed = min(MAX_SPEED, speed + 20)
        display.scroll(str(speed))
    elif value == 0x70:    # -
        speed = max(MIN_SPEED, speed - 20)
        display.scroll(str(speed))

    elif value == 0x40:    #light
        light_mode = (light_mode + 1) % 9
        if light_mode == 0:
            lb.all_lights_off()
        elif light_mode == 1:
            lb.all_lights_set_color(255, 0, 0)
        elif light_mode == 2:
            lb.all_lights_set_color(0, 255, 0)
        elif light_mode == 3:
            lb.all_lights_set_color(0, 0, 255)
        elif light_mode == 4:
            lb.all_lights_set_color(255, 255, 0)
        elif light_mode == 5:
            lb.all_lights_set_color(0, 255, 255)
        elif light_mode == 6:
            lb.all_lights_set_color(255, 0, 255)
        elif light_mode == 7:
            lb.all_lights_set_color(255, 255, 255)
        elif light_mode == 8:
            lb.all_lights_rainbow_cycle(2000)

    elif value == 0xA0: #Buzzer
        music.pitch(523, 250)
        music.pitch(659, 250)
        music.pitch(784, 250)
    # Servo control
    elif value == 0x08:   # 1
        servo_action = "S3+"
    elif value == 0x48:   # 3
```

```
        servo_action = "S3-"

    elif value == 0xA8:   # 5
        servo_action = "S2-"

    elif value == 0xB0:   # 0
        servo_action = "S2+"

    elif value == 0x28:   # 4
        servo_action = "S1-"
    elif value == 0x68:   # 6
        servo_action = "S1+"

    #  Posture preset action
    elif value == 0x88:   # 2
        S1_Angle = 90
        S2_Angle = 160
        S3_Angle = 90
        lb.set_servo('S1', 90)
        lb.set_servo('S2', 160)
        lb.set_servo('S3', 90)

    elif value == 0x18:   # 7
        servo_action = None
        lb.set_servo("S3", 90)
        sleep_ms(500)
        lb.set_servo("S1", 150)
        sleep_ms(500)
        lb.set_servo("S2", 160)
        s1, s2, s3 = 150, 160, 90

    elif value == 0x98:   # 8
        servo_action = None
        lb.set_servo("S3", 90)
        sleep_ms(500)
        lb.set_servo("S1", 90)
        sleep_ms(500)
        lb.set_servo("S2", 70)
        s1, s2, s3 = 90, 70, 90

    elif value == 0x58:   # 9
        servo_action = None
        lb.set_servo("S3", 90)
        sleep_ms(500)
        lb.set_servo("S2", 70)
        sleep_ms(500)
        lb.set_servo("S1", 90)
        s1, s2, s3 = 90, 70, 90

    #Power
    elif value == 0x00:
        lb.stop_motor()
        lb.all_lights_off()
        servo_action = None

    servo_control()
```

Experimental Phenomenon

Point the infrared remote control directly at the receiver probe. When a key on the infrared remote control is pressed, the car will perform different actions. Here we mainly explain that the "Buzzer" and "Light Control" functions and the number keys "2", "7", "8", "9" are the preset servo actions, which are triggered by pressing the keys.



Remote Control (Key Value)	Actual Function
0x00	Turn off the motor and lights
0x80	Control the car to move forward
0x40	Control the lights (switch the light effect each time it is pressed)
0x20	Control the car to turn left
0xA0	Control the buzzer
0x60	Control the car to turn right
0x10	Control the car to spin left
0x90	Control the car to move backward
0x50	Control the car to spin right
0x30	Control the car to speed up
0x70	Control the car to slow down
0xB0	Control the robotic arm to move up
0x08	Control the robotic arm to move left
0x88	Reset the robotic arm
0x48	Control the robotic arm to move right
0x28	Control the robotic arm to open the gripper
0xA8	Control the robotic arm to move down
0x68	Control the robotic arm to close the gripper
0x18	Grip the block and lift it up
0x98	Control the robotic arm to the gripping state
0x58	Put down the gripped block
0xFF	Default value